

Methods in Java

Rizwan Rashid

Opening Problem

- Find the sum of integers from 1 to 10, from 20 to 30, and from 35 to 45, respectively.

Problem

```
int sum = 0;
for (int i = 1; i <= 10; i++)
    sum += i;
System.out.println("Sum from 1 to 10 is " + sum);

sum = 0;
for (int i = 20; i <= 30; i++)
    sum += i;
System.out.println("Sum from 20 to 30 is " + sum);

sum = 0;
for (int i = 35; i <= 45; i++)
    sum += i;
System.out.println("Sum from 35 to 45 is " + sum);
```

Problem

```
int sum = 0;  
for (int i = 1; i <= 10; i++)  
    sum += i;
```

```
System.out.println("Sum from 1 to 10 is " + sum);
```

```
sum = 0;  
for (int i = 20; i <= 30; i++)  
    sum += i;
```

```
System.out.println("Sum from 20 to 30 is " + sum);
```

```
sum = 0;  
for (int i = 35; i <= 45; i++)  
    sum += i;
```

```
System.out.println("Sum from 35 to 45 is " + sum);
```

Solution

```
public static int sum(int i1, int i2) {  
    int sum = 0;  
    for (int i = i1; i <= i2; i++)  
        sum += i;  
    return sum;  
}
```

```
public static void main(String[] args) {  
    System.out.println("Sum from 1 to 10 is " + sum(1, 10));  
  
    System.out.println("Sum from 20 to 30 is " + sum(20, 30));  
  
    System.out.println("Sum from 35 to 45 is " + sum(35, 45));  
}
```

Methods

A method is a collection of statements grouped together to perform an operation

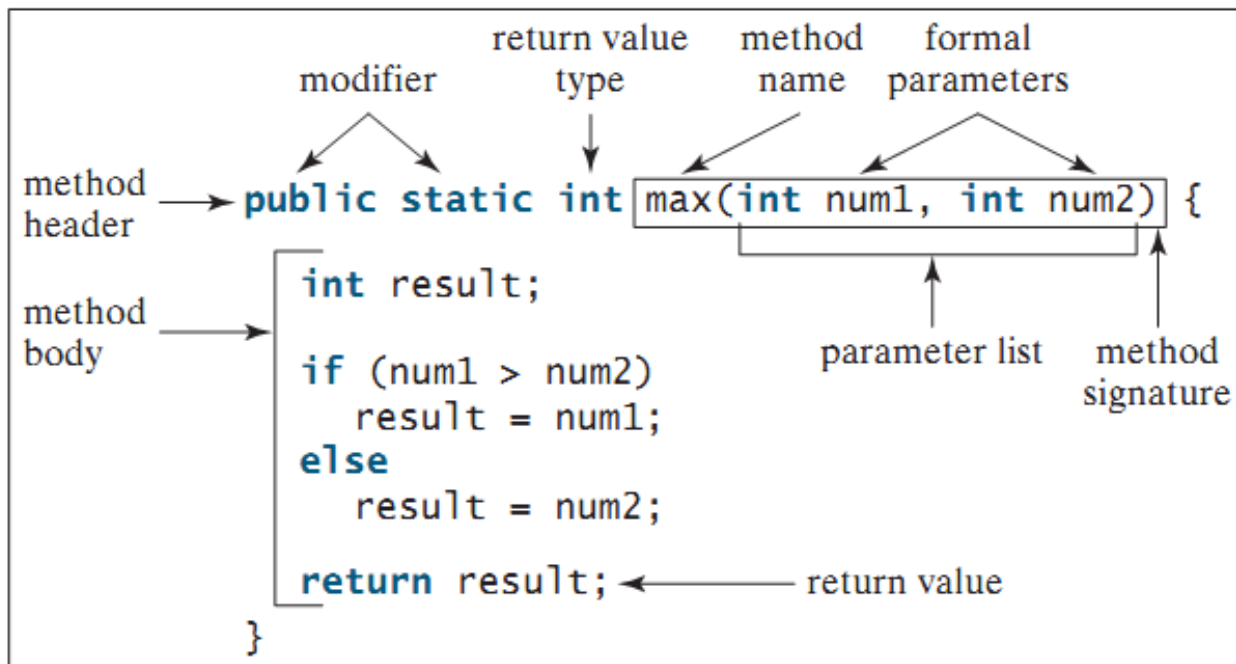
- Pre-defined Methods
 - Already defined in the Java class libraries known as **standard library method** or **built-in method**
 - `println()`, `Math.pow()`, `input.nextInt()`, etc.
- User-defined Methods
 - The method written by the user or programmer
 - Modified as per requirement

Defining a Method

- Syntax

```
modifier returnValueType methodName(list of parameters)
{
    // Method body;
}
```

Define a method



Invoke a method

```
int z = max(x, y);
```

Annotations and labels:

- actual parameters (arguments)**: Two arrows point to `x` and `y` in the argument list.

Defining a Method

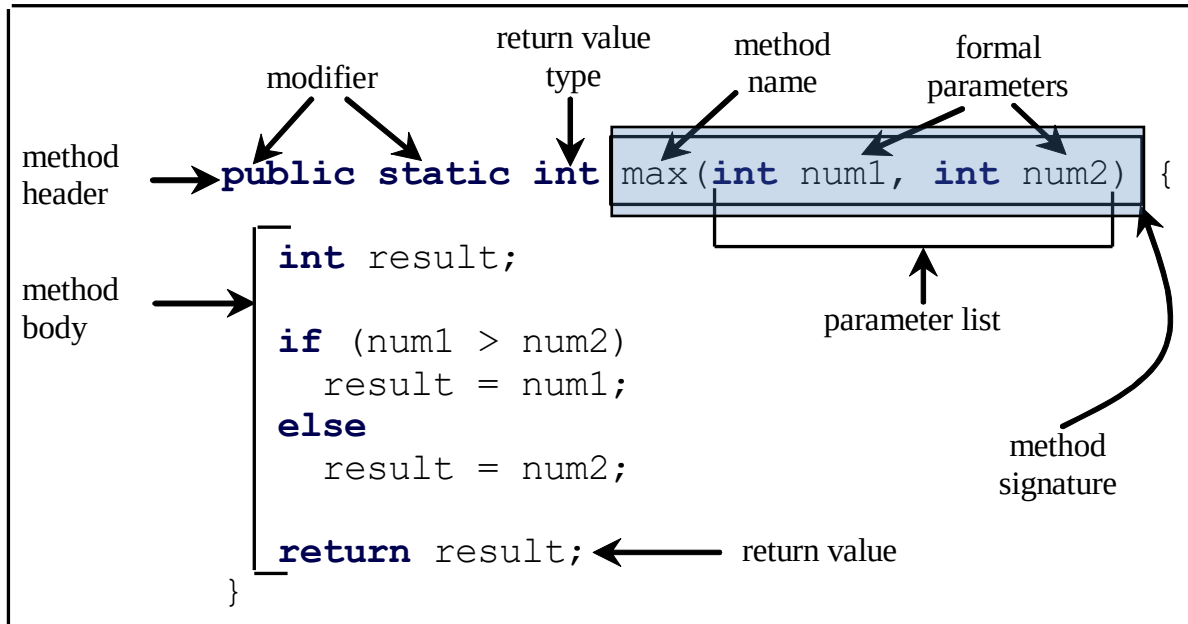
- **Access Specifier:**

- It specifies the visibility of the method. Java provides **four** types of access specifier:
- **Public:** The method is accessible by all classes
- **Private:** The method is accessible only in the classes in which it is defined
- **Protected:** The method is accessible within the same package or subclasses in a different package
- **Default:** When we do not use any access specifier in the method declaration, Java uses default access specifier by default. It is visible only from the same package only.

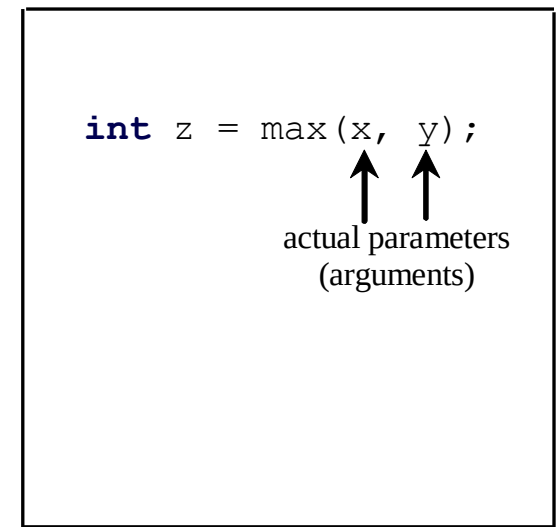
Method Signature

Method signature is the combination of the method name and the parameter list.

Define a method



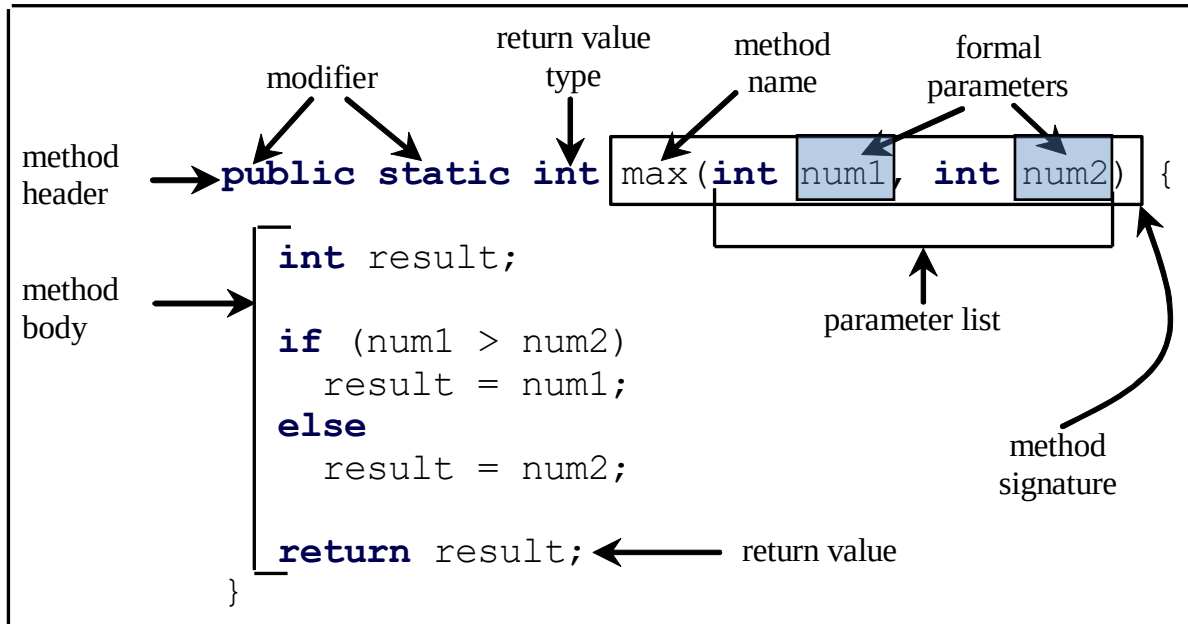
Invoke a method



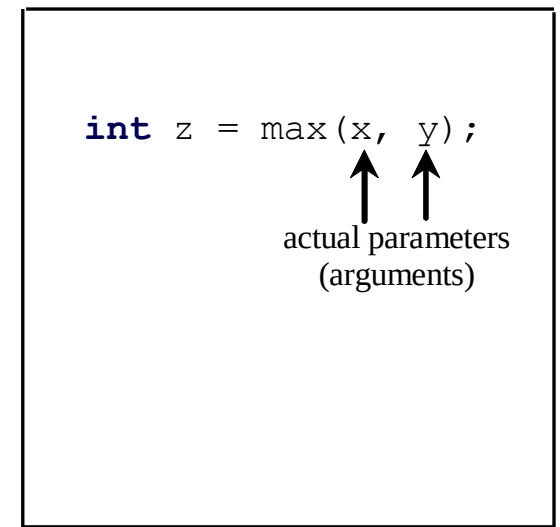
Formal Parameters

The variables defined in the method header are known as *formal parameters*.

Define a method



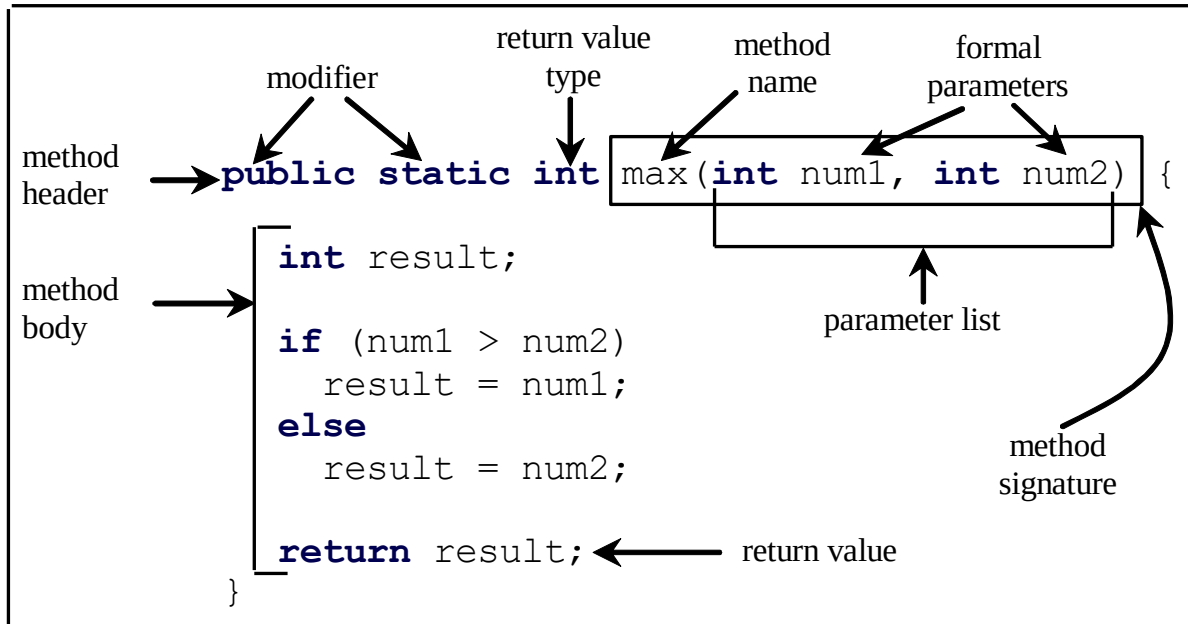
Invoke a method



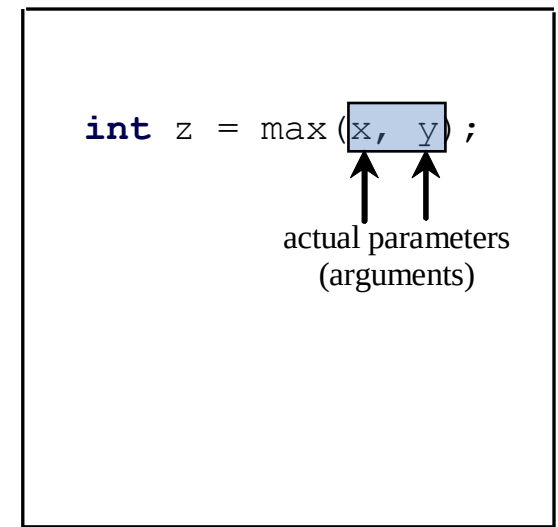
Actual Parameters

When a method is invoked, you pass a value to the parameter. This value is referred to as *actual parameter or argument*.

Define a method



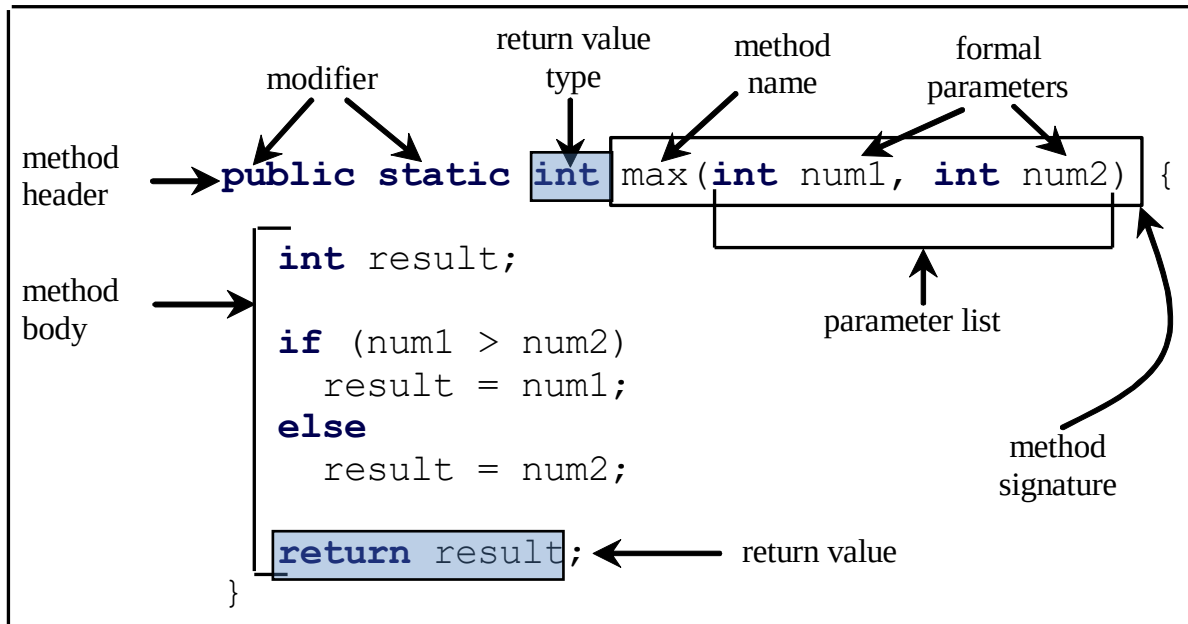
Invoke a method



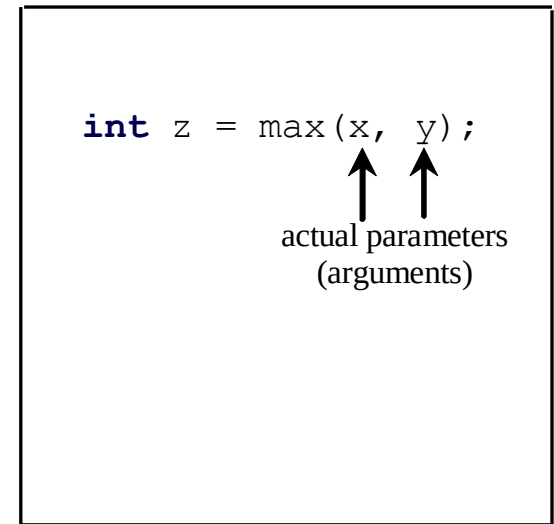
Return Value Type

A method may return a value. The returnValueType is the data type of the value the method returns. If the method does not return a value, the returnValueType is the keyword void. For example, the returnValueType in the main method is void.

Define a method



Invoke a method



Naming a Method

- **Single-word method name:** `sum()`, `area()`
- **Multi-word method name:** `areaOfCircle()`, `stringComparision()`
- It is also possible that a method has the same name as another method name in the same class, it is known as **method overloading**.

Java main() method

- The main() is the starting point for JVM to start execution of a Java program

public:

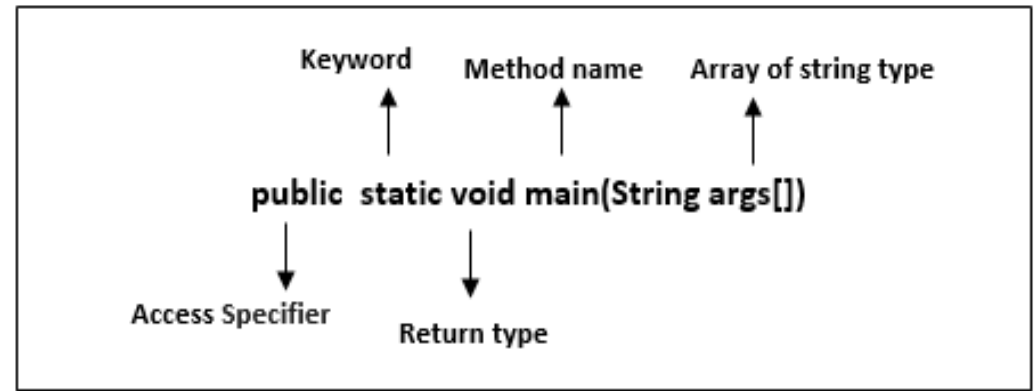
JVM can identify the execution point of the program. If we use private, protected, and default before the main() method, it will not be visible to JVM

static:

Static methods are the method which invokes without creating the objects, so we do not need any object to call the main() method.

void:

Void keyword acknowledges the compiler that main() method does not return any value.



main():

It is a default signature which is predefined in the JVM. It is called by JVM to execute a program line by line and end the execution after completion of this method. We can also overload the main() method.

String args[]:

The main() method also accepts some data from the user. It accepts a group of strings, which is called a string array. It is used to hold the command line arguments in the form of string values.

Java main() method

- What happens if the main() method is written without String args[]?
 - The program will compile, but not run, because JVM will not recognize the main() method
 - JVM always looks for the main() method with a string type array as a parameter.
- Execution Process
 1. First, JVM executes the static block
 2. Then it executes static methods
 3. Then it creates the object needed by the program
 4. Finally, it executes the instance methods

```
public static void main(String[] args){  
    System.out.println("Hello");  
}  
static          //static block  
{  
    System.out.println("Static block");  
}
```

Different way to write main()

- **public static void** main(String args[])
- **static public void** main(String args[])
- **static public void** main(String[] x)
- **static public void** main(String []x)
- **static public void** main(String...args)

Calling a Method

- Calling a method executes the code in the method
- **Method definition:** indicates what the method does
- How to call a method?

1. Value returning method: a call to the method is *treated as a value*

```
int larger = max(3, 4);
```

or

```
System.out.println(max(3, 4));
```

2. Void method: a call to the method must be a *statement*

```
System.out.println("Hello");
```

```
public static int max(int num1, int num2)
{
    int result;

    if(num1 > num2)
        result = num1;
    else
        result = num2;

    return result;
}
```

Calling a Method

- When a program calls a method, program control is transferred to the called method
- How is the control returned?
 - `return` statement is executed OR
 - Ending closing brace (`}`) is reached

Using/reusing a method

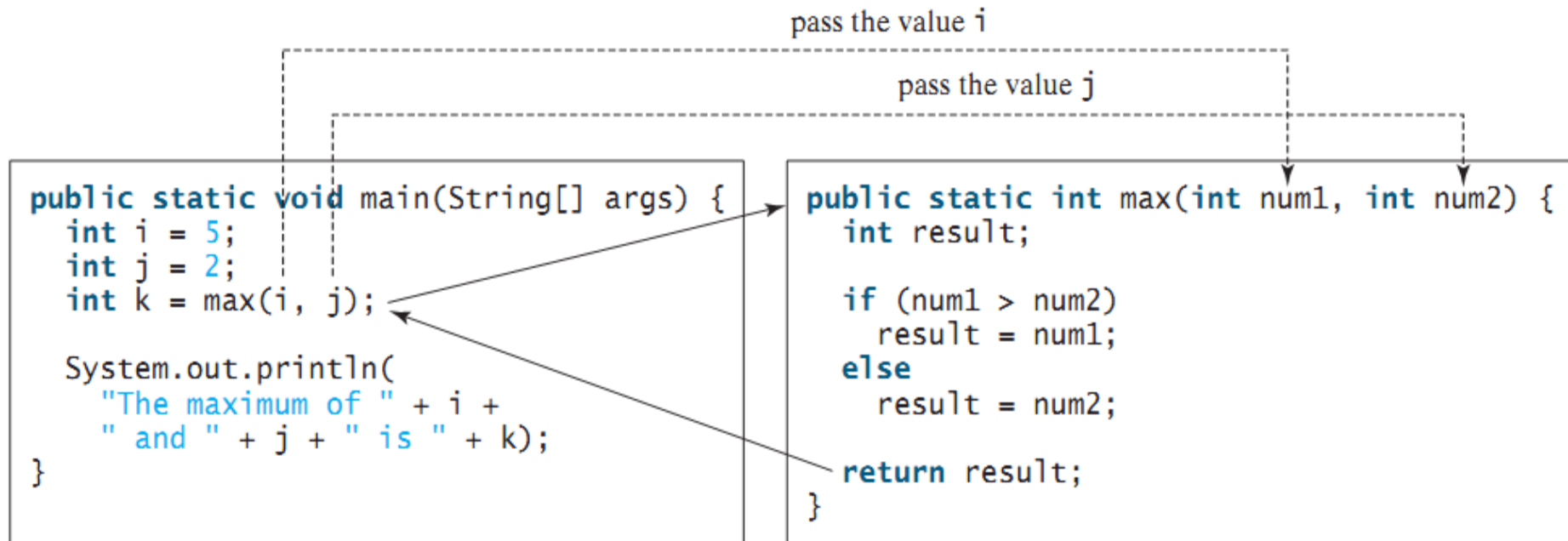
i	j	k	num1	num2	result
5	2				
			5	2	undefined
		5			5

The max of 5 and 2 is 5

Reusing a static method: **Classname.methodName**
Reusing max(): TestMax.max

```
public class TestMax {  
    /* main method */  
    public static void main(String[] args){  
        int i = 5;  
        int j = 2;  
        int k = max(i, j);  
        System.out.println("The max of "  
+ i + " and " + j + " is " + k);  
    }  
    /* Returns the maximum of two numbers */  
    public static int max(int num1, int  
num2){  
        int result;  
        if(num1 > num2)  
            result = num1;  
        else  
            result = num2;  
        return result;  
    }  
}
```

Calling a Method

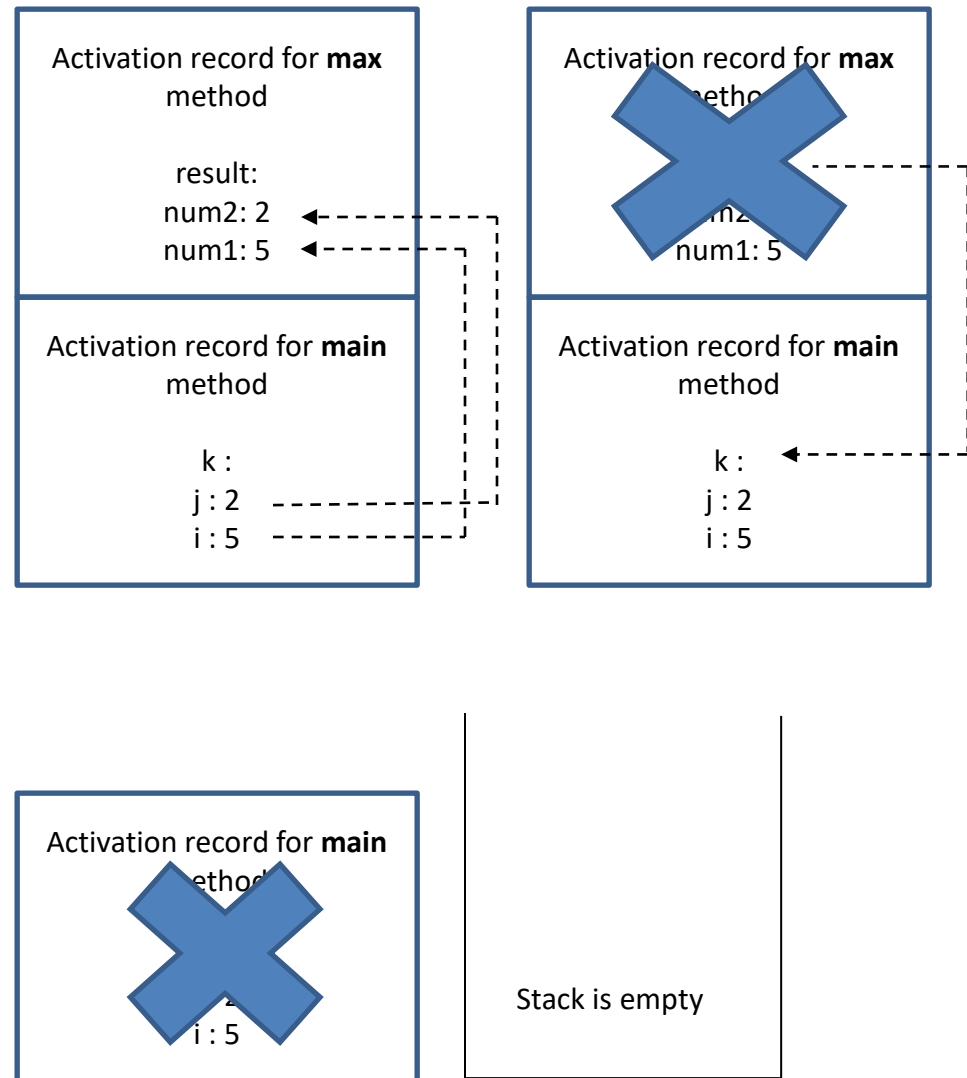


Activation record & call stack

- Each time a method is invoked, the system creates an **activation record**, *a.k.a activation frame*
 - Stores parameters and variables for the method
- The system places the record in an area of memory known as a **call stack** (*a.k.a execution stack, runtime stack, machine stack, or just stack*)
 - Information is stored in a **last-in, first-out** fashion
- When a method calls another method, the caller's activation record is kept intact, and a new activation record is created for the called method
- When the called method finishes its work and returns to its caller, its activation record is removed from the call stack

Call stack: example

```
public class TestMax {  
    /* main method */  
    public static void main(String[] args){  
        int i = 5;  
        int j = 2;  
        int k = max(i, j);  
        System.out.println("The max of "  
+ i + " and " + j + " is " + k);  
    }  
    /* Returns the maximum of two numbers */  
    public static int max(int num1, int  
num2){  
        int result;  
        if(num1 > num2)  
            result = num1;  
        else  
            result = num2;  
        return result;  
    }  
}
```



Void method

- A void method does not return a value.
- No return statement is used; ending brace returns the control to the caller method

The grade is C
The grade is F

```
1  public class TestVoidMethod {
2      public static void main(String[] args) {
3          System.out.print("The grade is ");
4          printGrade(78.5);
5
6          System.out.print("The grade is ");
7          printGrade(59.5);
8      }
9
10     public static void printGrade(double score) {
11         if (score >= 90.0) {
12             System.out.println('A');
13         }
14         else if (score >= 80.0) {
15             System.out.println('B');
16         }
17         else if (score >= 70.0) {
18             System.out.println('C');
19         }
20         else if (score >= 60.0) {
21             System.out.println('D');
22         }
23         else {
24             System.out.println('F');
25         }
26     }
27 }
```

Void method with return

```
public static void printGrade(double score) {  
    if (score < 0 || score > 100) {  
        System.out.println("Invalid score");  
        return;  
    }  
  
    if (score >= 90.0) {  
        System.out.println('A');  
    }  
    else if (score >= 80.0) {  
        System.out.println('B');  
    }  
    else if (score >= 70.0) {  
        System.out.println('C');  
    }  
    else if (score >= 60.0) {  
        System.out.println('D');  
    }  
    else {  
        System.out.println('F');  
    }  
}
```


Passing arguments by values

- The arguments are passed by value to parameters when invoking a method (pass-by-value)
 - If the argument is a variable, the value of the variable is passed to the parameter
- Parameter order association
 - Arguments must be in the same order as their respective parameters in the method signature
- Example:
 - nPrintln("Hi", 3);
 - nPrintln(3, "Hi");
 - nPrintln("Hi", '3');

```
public static void nPrintln(String message, int n) {  
    for (int i = 0; i < n; i++)  
        System.out.println(message);  
}
```

Pass-by-value (example 1)

```
1 public class Increment {  
2     public static void main(String[] args) {  
3         int x = 1;  
4         System.out.println("Before the call, x is " + x);  
5         increment(x);  
6         System.out.println("After the call, x is " + x);  
7     }  
8  
9     public static void increment(int n) {  
10        n++;  
11        System.out.println("n inside the method is " + n);  
12    }  
13 }
```

```
Before the call, x is 1  
n inside the method is 2  
After the call, x is 1
```

Pass-by-value (example 2)

```
1 public class TestPassByValue {  
2     /** Main method */  
3     public static void main(String[] args) {  
4         // Declare and initialize variables  
5         int num1 = 1;  
6         int num2 = 2;
```

Before invoking the swap method, num1 is 1 and num2 is 2

Inside the swap method

Before swapping, n1 is 1 and n2 is 2

After swapping, n1 is 2 and n2 is 1

After invoking the swap method, num1 is 1 and num2 is 2

```
14         System.out.println("After invoking the swap method, num1 is " +  
15             num1 + " and num2 is " + num2);  
16     }
```

What will happen if we change the parameter name **n1** in **swap** method to **num1**?

```
27         n2 = temp;  
28  
29         System.out.println("\t\tAfter swapping, n1 is " + n1  
30             + " and n2 is " + n2);  
31     }  
32 }
```

Overloading methods

- Overloading the methods enables their redefinition with the same name
- Signatures of overloaded methods must be different
- Different number or type of parameters

```
/** Return the max of two numbers */  
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

```
int larger = max(10, 5);
```

```
public static double max(double num1, double num2) {  
    if (num1 > num2)  
        return num1;  
    else  
        return num2;  
}
```

```
double larger = max(12.3, 15.7);
```

```

1 public class TestMethodOverloading {
2     /** Main method */
3     public static void main(String[] args) {
4         // Invoke the max method with int parameters
5         System.out.println("The maximum of 3 and 4 is "
6             + max(3, 4));
7
8         // Invoke the max method with the double parameters
9         System.out.println("The maximum of 3.0 and 5.4 is "
10             + max(3.0, 5.4));
11
12         // Invoke the max method with three double parameters
13         System.out.println("The maximum of 3.0, 5.4, and 10.14 is "
14             + max(3.0, 5.4, 10.14));
15     }
16

```

The maximum of 3 and 4 is 4
The maximum of 3.0 and 5.4 is 5.4
The maximum of 3.0, 5.4, and 10.14 is 10.14

max(2, 2.5); ????

Which max method will be invoked?

```

17 /** Return the max of two int values */
18 public static int max(int num1, int num2) {
19     if (num1 > num2)
20         return num1;
21     else
22         return num2;
23 }
24

```

```

25 /** Find the max of two double values */
26 public static double max(double num1, double num2) {
27     if (num1 > num2)
28         return num1;
29     else
30         return num2;
31 }
32

```

```

33 /** Return the max of three double values */
34 public static double max(double num1, double num2, double num3) {
35     return max(max(num1, num2), num3);
36 }
37 }

```

Why max(double, double) is not invoked for the call max(3, 4)?

Ambiguous invocation of overloaded method

- Two or more possible matches for the invocation
- Best match cannot be determined
- Generates a compiler error

```
public class AmbiguousOverloading {  
    public static void main(String[] args) {  
        System.out.println(max(1, 2));  
    }  
  
    public static double max(int num1, double num2) {  
        if (num1 > num2)  
            return num1;  
        else  
            return num2;  
    }  
  
    public static double max(double num1, int num2) {  
        if (num1 > num2)  
            return num1;  
        else  
            return num2;  
    }  
}
```

Scope of variables

- The scope of a variable is the portion of the program where it can be referenced
- Local variable: a variable defined inside a method
 - Scope starts from its declaration; continues to the end of the block
 - Must be declared and assigned a value before its use
 - **Method-level scope**
 - Parameter is a local variable: scope is within the entire method body
 - Variables inside a method have method level scope
 - **Block-level scope**
 - A variable declared in header of a for-loop can be referenced in the entire loop
 - A variable declared inside for-loop's body has its scope after declaration till the end of loop

```

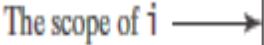
public class Test {
    public static void
main(String[] args){
    int x = 5;
    System.out.println(x);
    System.out.println(y);
}
public static void Method1(){
    int x = 1;
    int y = 10;
    System.out.println(y);
    System.out.println(z);
}
public static void Method2(){
    int x = 13;
    int y = 12;
    int z = 5;
    System.out.println(x);
    System.out.println(y);
}
}

```

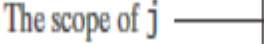
```

public static void method1() {
    .
    .
    for (int i = 1; i < 10; i++) {
        .
        .
        int j;
        .
        .
    }
}

```



The scope of i



The scope of j

Scope of variables

It is fine to declare `i` in two nonnested blocks.

```
public static void method1() {  
    int x = 1;  
    int y = 1;  
  
    for (int i = 1; i < 10; i++) {  
        x += i;  
    }  
  
    for (int i = 1; i < 10; i++) {  
        y += i;  
    }  
}
```

It is wrong to declare `i` in two nested blocks.

```
public static void method2() {  
    int i = 1;  
    int sum = 0;  
  
    for (int i = 1; i < 10; i++)  
        sum += i;  
}
```

A common mistake

```
for (int i = 0; i < 10; i++) {  
      
    System.out.println(i);  
}
```

Summary

- Method definition & calling
- Value returning method & void method
- The call stack
- Transfer of control
- Pass-by-value (argument passing)
- Method overloading
- The scope of variables